

How to Enable Hibernate Statistics to Analyze Performance

👤 Givantha Kalansuriya 📁 Hibernate, performance ⌵ 1 Minute

i

1 Vote

In Java world most of the developer have use Hibernate as ORM Framework in their applications development and most of them have encountering the performance issues. The beauty is most of the time performance issues are visible when application push to production.

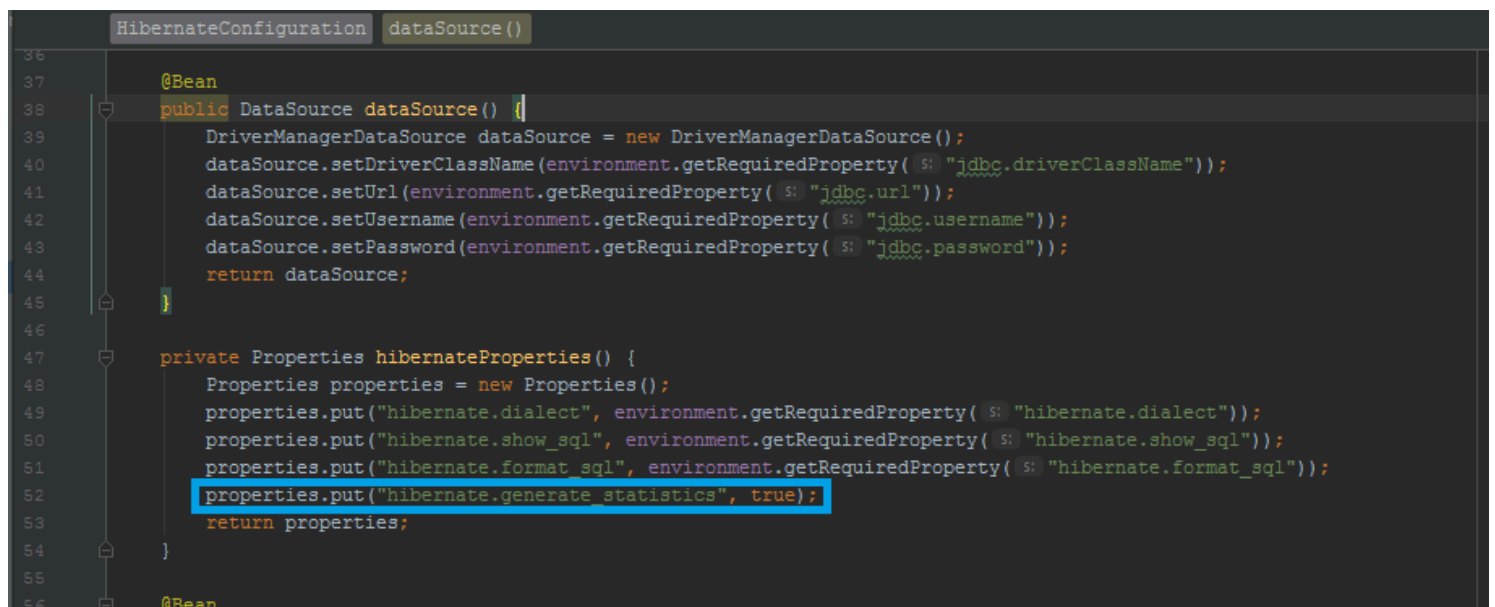
According to my experience most of the performance drops are often related to slow database query at that time we need to find which queries causes the problem ? 😞

Don't worry there is a easy way to find that issue. In hibernate can be configure to collect statistics and provide statistics to analyze database query issues.

Let's see how we can do it.

Step 1. Enable hibernate.generate_statistics

```
properties.put("hibernate.generate_statistics", true);
```



```
36
37
38 @Bean
39 public DataSource dataSource() {
40     DriverManagerDataSource dataSource = new DriverManagerDataSource();
41     dataSource.setDriverClassName(environment.getRequiredProperty("${jdbc.driverClassName}"));
42     dataSource.setUrl(environment.getRequiredProperty("${jdbc.url}"));
43     dataSource.setUsername(environment.getRequiredProperty("${jdbc.username}"));
44     dataSource.setPassword(environment.getRequiredProperty("${jdbc.password}"));
45     return dataSource;
46 }
47
48 private Properties hibernateProperties() {
49     Properties properties = new Properties();
50     properties.put("hibernate.dialect", environment.getRequiredProperty("${hibernate.dialect}"));
51     properties.put("hibernate.show_sql", environment.getRequiredProperty("${hibernate.show_sql}"));
52     properties.put("hibernate.format_sql", environment.getRequiredProperty("${hibernate.format_sql}"));
53     properties.put("hibernate.generate_statistics", true);
54     return properties;
55 }
56
57 @Bean
```

Step 2. Hibernate Logs enable and set level to debug (This is not required)

```
log4j.properties x
1 # Root logger option
2 log4j.rootLogger=DEBUG, stdout, file
3
4 # Redirect log messages to console
5 log4j.appender.stdout=org.apache.log4j.ConsoleAppender
6 log4j.appender.stdout.Target=System.out
7 log4j.appender.stdout.layout=org.apache.log4j.PatternLayout
8 log4j.appender.stdout.layout.ConversionPattern=%d{yyyy-MM-dd HH:mm:ss} %-5p %c{1}:%L - %m%n
9
10 # Redirect log messages to a log file, support file rolling.
11 log4j.appender.file=org.apache.log4j.RollingFileAppender
12 log4j.appender.file.File=C:\\log4j-application.log
13 log4j.appender.file.MaxFileSize=5MB
14 log4j.appender.file.MaxBackupIndex=10
15 log4j.appender.file.layout=org.apache.log4j.PatternLayout
16 log4j.appender.file.layout.ConversionPattern=%d{yyyy-MM-dd HH:mm:ss} %-5p %c{1}:%L - %m%n
17
18 #hibernate logs enable debug
19 log4j.logger.org.hibernate.SQL=debug
20 log4j.logger.org.hibernate.type.descriptor.sql=trace
```

After these two steps you can run your code and refer the console logs for access the hibernate session statics. In there I'm used sample application for save 3 employee object and retrieve all employee from database.

```
25 * Create Employee1
26 */
27 Employee employee1 = new Employee();
28 employee1.setName("Name" + 1);
29 employee1.setJoiningDate(new LocalDate( R 2012, i1: 10, i2: 10));
30 employee1.setSalary(new BigDecimal( val: 10000));
31 employee1.setSsn("ssn000000033142");
32
33 Employee employee2 = new Employee();
34 employee2.setName("Name" + 2);
35 employee2.setJoiningDate(new LocalDate( R 2010, i1: 10, i2: 10));
36 employee2.setSalary(new BigDecimal( val: 10000));
37 employee2.setSsn("ssn00100000232422");
38
39 Employee employee3 = new Employee();
40 employee3.setName("Name" + 3);
41 employee3.setJoiningDate(new LocalDate( R 2013, i1: 10, i2: 10));
42 employee3.setSalary(new BigDecimal( val: 10000));
43 employee3.setSsn("ssn000002300323422");
44
45 * Persist both Employees
46 */
47 service.saveEmployee(employee1);
48 service.saveEmployee(employee2);
49 service.saveEmployee(employee3);
50
51
52 /*
53 * Get all employees list from database
54 */
55 List<Employee> employeeList = service.findAllEmployees();
56 for (Employee emp : employeeList) {
57     logger.info(emp.toString());
58 }
59
```

Step 3. Access logs to read hibernate statics details.

```
7
May 05, 2017 11:45:25 AM org.hibernate.engine.internal.StatisticalLoggingSessionEventListener and
INFO: Session Metrics {
  12447003 nanoseconds spent acquiring 1 JDBC connections;
  0 nanoseconds spent releasing 0 JDBC connections;
  235244 nanoseconds spent preparing 1 JDBC statements;
  2810203 nanoseconds spent executing 1 JDBC statements;
  0 nanoseconds spent executing 0 JDBC batches;
  0 nanoseconds spent performing 0 L2C puts;
  0 nanoseconds spent performing 0 L2C hits;
  0 nanoseconds spent performing 0 L2C misses;
  307911 nanoseconds spent executing 1 flushes (flushing a total of 3 entities and 0 collections);
  59529 nanoseconds spent executing 1 partial-flushes (flushing a total of 0 entities and 0 collections)
}
May 05, 2017 11:45:25 AM com.websystique.spring.AppMain main
INFO: Employee [id=1, name=Name1, joiningDate=2012-10-10, salary=10000, ssn=ssn000000033142]
May 05, 2017 11:45:25 AM com.websystique.spring.AppMain main
INFO: Employee [id=2, name=Name2, joiningDate=2010-10-10, salary=10000, ssn=ssn00100000232422]
May 05, 2017 11:45:25 AM com.websystique.spring.AppMain main
INFO: Employee [id=3, name=Name3, joiningDate=2013-10-10, salary=10000, ssn=ssn000002300323422]
May 05, 2017 11:45:25 AM org.springframework.context.annotation.AnnotationConfigApplicationContext doClose
INFO: Closing org.springframework.context.annotation.AnnotationConfigApplicationContext@2ef1e4fa: startup date [Fri May 05 11:45:21 NPT 2017]; root of context hierarchy
```

Summary

In java applications to find performance issues, we can use hibernate statistics this configuration we can use development stage to identify the hibernate performance issue. Based on that we can easily identify the application issues in which sections.

Hope this article helps to identify your application hibernate performance issue.

Tagged:

Hibernate,
Java,
performance,
statistics

Published by Givantha Kalansuriya



[View all posts by Givantha Kalansuriya](#)

[Website Built with WordPress.com.](#)